App Dev Project Report

1. Student Details

Name: Vishesh Sahu
Roll Number: 23F3001373
Email: 23f3001373@ds.study.iitm.ac.in
About Me: I am a diploma level student of IITM BS Degree in Data Science and Applications.

2. Project Details

Project Title: Placement Portal App

Problem Statement:

To design and build a multi-role web application that streamlines campus placement processes by connecting Students, Companies, and an Administrator. The portal allows companies to post placement drives, students to apply for openings, and the admin to manage approvals, user accounts, and overall placement activity.

Approach:

The application was built using Flask as the backend framework with a role-based architecture. It supports three user roles — Admin, Company, and Student — each with a dedicated dashboard and set of functionalities. SQLite was used as the database via SQLAlchemy ORM. The frontend was built with Jinja2 templates and Bootstrap 5 for responsive design. Flask-Login handles user authentication and session management, while Werkzeug provides secure password hashing.

3. AI/LLM Declaration

During the development, LLM (Google's Gemini) were utilized strictly as a supplementary learning tool, debugging assistant, and syntax reference. The AI was used to accelerate the learning process and overcome technical roadblocks.

The extent of AI/LLM usage is around 10–15%, limited to assist with syntax and troubleshooting.

All final implementation logic, debugging, and integration were done manually.

The core application logic, architectural decisions, database schema, and final code assembly were entirely directed and executed by me. I possess a complete and thorough understanding of the submitted codebase and can independently explain, modify, and execute all functionalities.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework (20%)
SQLAlchemy (Flask-SQLAlchemy)	Object Relational Mapper for SQLite database (15%)
Jinja2	Template engine for rendering dynamic HTML pages (15%)
Bootstrap 5 / CSS	Frontend styling and responsive design (8%)
JavaScript / AJAX	Client-side interactivity and form handling (5%)
Flask-Login	User authentication and session management (10%)
Werkzeug	Secure password hashing and file upload utilities
SQLite	Lightweight local database for storing all application data

5. Database Schema / ER Diagram

Tables:

User — stores user profile details for all roles (id, username, password, role, name, department, resume_filename, company_name, hr_contact, website, industry, is_approved, is_blacklisted)

Drive — stores placement drive details posted by companies (id, company_id, job_title, job_description, eligibility_criteria, application_deadline, status)

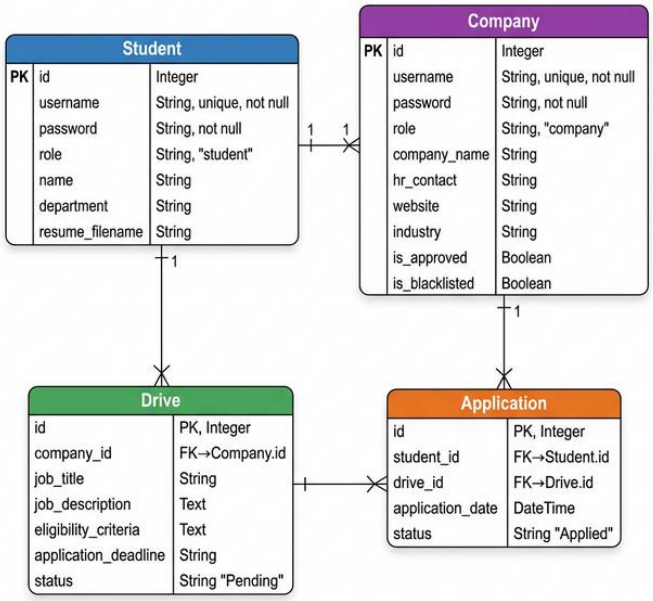
Application — stores student applications to placement drives (id, student_id, drive_id, application_date, status)

Relationships:

One-to-Many → User (Company) → Drive (via company_id)

One-to-Many → User (Student) → Application (via student_id)

One-to-Many → Drive → Application (via drive_id)



6. API Resource Endpoints

Endpoint	Method	Description
/	GET/POST	Login page — authenticate user and redirect by role
/register	GET/POST	Register new student or company account
/logout	GET	Log out the current user
/admin_dashboard	GET	Admin dashboard with stats, pending approvals, search
/admin/history	GET	View all drives and applications history
/approve_company/<id>	POST	Approve a pending company account
/reject_company/<id>	POST	Reject and remove a company account
/approve_drive/<id>	POST	Approve a pending placement drive
/reject_drive/<id>	POST	Reject a placement drive
/admin/blacklist/<id>	POST	Toggle blacklist status for a user
/admin/delete/<id>	POST	Delete a user and cascade-remove related data
/admin/edit_user/<id>	GET/POST	Edit student or company details
/company_dashboard	GET	Company dashboard showing

		all drives
/edit_company_profile	GET/POST	Edit company profile details
/create_drive	GET/POST	Create a new placement drive
/edit_drive/<id>	GET/POST	Edit an existing drive
/close_drive/<id>	POST	Close a placement drive
/delete_drive/<id>	POST	Delete a drive and its applications
/drive/<id>/applications	GET	View applications for a specific drive
/update_application/<id>	POST	Update application status (Shortlisted/Selected/Rejected)
/student_dashboard	GET	Student dashboard with available drives and applications
/edit_profile	GET/POST	Edit student profile and upload resume
/apply/<drive_id>	POST	Apply to a placement drive

7. Architecture and Features

Structure:

```
placement_portal/
├── app.py                  # Main Flask application with all routes
├── models.py               # SQLAlchemy database models (User, Drive, Application)
├── create_admin.py         # Standalone script to create admin user
├── requirements.txt        # Python dependencies
├── README.md
├── instance/
│   └── placement_portal.sqlite3   # SQLite database (auto-generated)
├── static/
│   └── resumes/            # Uploaded student resumes
└── templates/
    ├── base.html               # Base template with Bootstrap & flash messages
    ├── login.html              # Login page
    ├── register.html           # Registration form (Student / Company)
    ├── admin_dashboard.html    # Admin dashboard with stats & management
    ├── admin_edit_user.html    # Admin edit user details
    ├── admin_history.html      # Historical placement data
    ├── company_dashboard.html  # Company dashboard with drives
    ├── create_drive.html       # Create new placement drive
    ├── edit_drive.html         # Edit existing drive
    ├── drive_applications.html # View applications for a drive
    ├── edit_company_profile.html # Company profile editing
    ├── edit_profile.html       # Student profile editing
    └── student_dashboard.html  # Student dashboard with drives & history
```

Implemented Features:

- Multi-role authentication system (Admin, Company, Student)
- Role-based dashboards with dedicated functionalities
- Company registration with Admin approval workflow
- Placement drive creation, editing, closing, and deletion by companies
- Drive approval/rejection by Admin
- Student application to approved placement drives
- Application status tracking (Applied → Shortlisted → Selected/Rejected)
- Student profile management with resume upload (PDF, max 2MB)
- Company profile management
- Admin user management (edit, delete, blacklist/unblacklist)
- Search functionality across Admin and Student dashboards
- Admin history view for all drives and applications
- Cascade deletion of related data when users/drives are removed
- Secure password hashing using Werkzeug (PBKDF2-SHA256)
- Blacklist enforcement preventing login and drive applications
- Responsive UI built with Bootstrap 5
- Flash messaging for user feedback
- Duplicate application prevention
- Programmatic database creation on app startup
- Auto-creation of default admin account

8. Video Presentation

Drive Link:

https://drive.google.com/file/d/1i0r5w79i_J9-1oTl7T_L6kRhMAt6LV9w/view?usp=sharing